New Researcher Handbook

A Practical Guide for Early-Career PhD Students and Researchers

Xiaolong Luo
Harvard University
School of Engineering and Applied Sciences
Also available on: Xiaohongshu (小红书)

October 23, 2026

Contents

1 Introduction **4**
1.1 Why This Manual Matters . . . 4
1.2 How We Build Research Excellence . . . 5

2 Core Principles **5**
2.1 Clarity Above All . . . 5
2.2 Consistency Matters . . . 5
2.3 Respect Your Audience . . . 5

3 Presentation Standards **5**
3.1 Structure . . . 5
3.2 Slide Design Guidelines . . . 6

4 Experimental Results Presentation **6**
4.1 Experimental Setup Checklist . . . 6
4.2 Figure Guidelines . . . 6
4.2.1 Essential Elements . . . 6
4.2.2 Scientific Color Schemes . . . 7

5 Research Habits for CS & AI **7**
5.1 Daily Practices . . . 7
5.1.1 Code and Experiment Management . . . 7
5.1.2 Experimental Reproducibility: The Foundation of Credible Research . . . 8
5.1.3 Environment Management with Conda . . . 8
5.1.4 Containerization with Docker . . . 9
5.1.5 Working with Remote Servers and HPC Clusters . . . 11
5.1.6 Time Management . . . 15
5.2 Research Methodology . . . 15
5.2.1 Literature Review . . . 15
5.2.2 Research Idea Generation . . . 16
5.2.3 Experiment Design . . . 17
5.3 Collaboration Best Practices . . . 17
5.3.1 Communication . . . 17
5.3.2 Code Collaboration . . . 17

5.3.3 Active Participation in Group Meetings . . . 18
5.4 Professional Development . . . 18
5.4.1 Skills to Develop . . . 18

6 Report Writing Standards **18**
6.1 Structure . . . 18
6.2 Academic Writing Style . . . 19
6.3 Common Mistakes to Avoid . . . 20

7 Weekly Update Format **20**
7.1 Structure . . . 20
7.2 Best Practices . . . 20
7.3 Effective Communication with Your Advisor . . . 20

8 How to be a Productive PhD Student **22**
8.1 Strategic Research Framework . . . 22
8.1.1 Clear Goals and Structured Output Paths . . . 22
8.1.2 Daily Action System . . . 22
8.1.3 Personal Reflection — Navigating Mode Switching in Research Life . . . 23
8.2 Top Conferences by Field . . . 24
8.2.1 AI/Machine Learning Conferences . . . 24
8.2.2 Computer Science Conferences . . . 25
8.2.3 AI for Science Conferences and Venues . . . 25
8.3 Publication Strategy . . . 26
8.3.1 Conference Timeline Management . . . 26
8.3.2 Quality vs. Quantity Balance . . . 26
8.3.3 Collaboration Guidelines . . . 27
8.3.4 Technical Paper Planning: Key Points to Think Through . . . 27
8.3.5 Writing Effective Survey/Review Papers . . . 29
8.4 Productivity Systems and Tools . . . 31
8.4.1 Research Pipeline Management . . . 31
8.4.2 Time Management Strategies . . . 31
8.4.3 AI Tools for PhD Research . . . 32
8.4.4 Mental Health and Sustainability . . . 35
8.4.5 Understanding My Cognitive Mode Switching and Rest . . . 36
8.4.6 My Struggle with the Planning Fallacy . . . 37
8.5 Health, Friends, and Professional Support . . . 39
8.5.1 Your Body Is Infrastructure, Not a Reward . . . 40
8.5.2 Friends Are Part of the Infrastructure . . . 40
8.5.3 Talking to a Professional . . . 41
8.6 Academic Social Networking . . . 42
8.6.1 Building Your Research Network . . . 42
8.6.2 Online Presence . . . 43
8.6.3 Building Your Research Brand Online . . . 44
8.6.4 Building Your Academic Website . . . 45
8.7 Essential Reading for PhD Students . . . 46
8.7.1 Core PhD Experience . . . 47
8.7.2 Career and Professional Development . . . 47
8.7.3 Online Resources and Blogs . . . 47
8.8 Resource Channels Worth Your Attention . . . 48
8.8.1 Papers and Research News . . . 48
8.8.2 Long-Form Interviews and Podcasts . . . 48

8.8.3 Technical Blogs and Learning by Doing 49

A Presentation Checklist **50**

B Figure Checklist **50**

C Research Tools and Resources **50**
C.1 Recommended Software 50
C.2 Terminal Setup Recommendation 51
C.3 Visualization and Diagramming Tools 51
C.3.1 Excalidraw 51
C.3.2 PlotNeuralNet 52
C.4 System Resource Monitoring and Performance Optimization 52
C.4.1 System Resource Monitoring 52
C.4.2 Parallel Processing for Efficiency 53
C.4.3 Best Practices and Monitoring Guidelines 53

1 Introduction

1.1 Why This Manual Matters

Dear students,

I wish I had seen this manual when I was an undergraduate student, or when I just started my PhD journey. It would have saved me from countless detours, mistakes, and unnecessary struggles. This guide contains the lessons I learned the hard way—through trial and error, late-night debugging sessions, rejected papers, and awkward presentations.

Looking back, I realize how much time and energy I could have saved if someone had told me:

- The importance of version control (I lost weeks of work because I didn't use Git properly)
- How to structure presentations (my first conference talk was a disaster—no clear message, too much detail)
- The value of documentation (I couldn't understand my own code from 6 months earlier)
- How to ask good questions in meetings (I stayed silent for months, missing valuable learning opportunities)

These aren't just rules—they're shortcuts to success that I discovered too late:

- Clear communication skills will help you share your ideas effectively
- Rigorous research habits will make you a trusted and reliable researcher
- Professional presentation skills will open doors to opportunities
- Good documentation practices will save you countless hours in the future
- Collaborative skills will make you a valued team member anywhere you go

As a PhD candidate in my fourth year, I'm still learning these lessons myself. Every guideline in this manual comes from mistakes I've made or witnessed firsthand. When I emphasize the importance of presentation skills, it's because I've missed many opportunities and left poor impressions due to bad presentations. When I emphasize documentation, it's because I've personally struggled to recreate my own experiments from months earlier.

The habits we develop now will serve us when we're:

- Presenting at international conferences (where we have one shot to impress)
- Defending our theses (where clarity can make or break the defense)
- Interviewing for our dream jobs (where these skills set us apart)
- Eventually leading our own research teams (where we'll pass on these lessons)
- Pitching ideas to investors or stakeholders (where professionalism matters most)

My intention is simple: By sharing what I've learned so far in my PhD journey, I hope we can learn from each other's experiences. This manual isn't written from a position of mastery—it's a collection of lessons I'm still learning, mistakes I'm still making, and habits I'm still building. Let's navigate this journey together and help each other become the researchers we aspire to be.

1.2 How We Build Research Excellence

Beyond the technical skills, remember that excellence in research comes from:

- **Frequent feedback from each other**: Don't work in isolation—share your work early and often
- **Reflecting on our personal and research practices**: Take time to think about what's working and what isn't
- **Getting enough sleep so we can bring our best selves to our work**: Exhausted researchers make poor decisions and write buggy code

These principles might sound simple, but I've seen too many brilliant students burn out because they ignored them. Your best work comes when you're well-rested, connected with your community, and continuously learning from both successes and failures.

2 Core Principles

2.1 Clarity Above All

Every presentation should prioritize clarity. Your audience should understand:

- The problem you're solving
- Your approach
- Your results and their implications

2.2 Consistency Matters

Use consistent formatting, terminology, and structure throughout your presentations and reports.

2.3 Respect Your Audience

Remember that your audience's time is valuable. Be prepared, be concise, and be engaging.

3 Presentation Standards

3.1 Structure

Every presentation should follow this structure:

1. **Title Slide**: Include title, your name, date, and affiliation
2. **Outline**: Brief overview of what you'll cover
3. **Background/Motivation**: Why this work matters
4. **Problem Statement**: Clear definition of what you're solving
5. **Methodology**: Your approach
6. **Results**: What you found
7. **Discussion**: What it means
8. **Future Work**: Next steps
9. **Acknowledgments**: Credit collaborators and funding

3.2 Slide Design Guidelines

- **6×6 Rule**: Maximum 6 bullet points with 6 words each
- **Font Size**: Minimum 24pt for body text, 32pt for titles
- **Color Scheme**: Use high contrast; avoid red-green combinations
- **One Message Per Slide**: Each slide should convey one main idea

4 Experimental Results Presentation

Critical Requirements

Before presenting ANY experimental results:

1. Clearly explain the experimental setting
2. Define all variables and parameters
3. State your hypotheses explicitly

Before showing ANY figure:

1. Explain what the figure shows
2. Describe the X and Y axes clearly
3. Read the caption aloud
4. Point out key trends or findings

4.1 Experimental Setup Checklist

When presenting experiments, always include:

- **Dataset**: Size, source, preprocessing steps
- **Baselines**: What you're comparing against
- **Metrics**: How you measure success
- **Hyperparameters**: All relevant settings
- **Statistical Significance**: Error bars, p-values when applicable

4.2 Figure Guidelines

4.2.1 Essential Elements

Every figure MUST have:

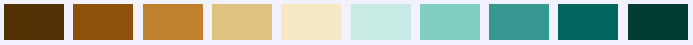
- Clear, descriptive title and Labeled axes with units
- Legend (if multiple data series)
- Caption explaining what's shown
- Appropriate scale and range

- Use vector graphics (PDF, SVG) when possible
- Include error bars or confidence intervals

4.2.2 Scientific Color Schemes

Professional Color Palettes

Recommended Scientific Color Schemes:

1. **Sequential (Single Hue):** For ordered data
   - **Blues:** 
   - **Viridis:** 
   - **Cividis:** 
2. **Diverging:** For data with meaningful midpoint
   - **RdBu (Red-Blue):** 
   - **PuOr (Purple-Orange):** 
   - **BrBG (Brown-Green):** 
3. **Categorical:** For discrete categories
   - **Set1:** 
   - **Okabe-Ito (Colorblind-safe):** 
   - **Dark2:** 

Color Selection Resource:

- Best Color Palettes for Scientific Figures and Data Visualizations - Comprehensive guide for scientific color selection

Note: Multi-Subplot Consistency

When creating figures with multiple subplots comparing different models, always use the same color and ordering for each model across all subplots. For example, if XGBoost is blue in subplot 1, it must be blue in all other subplots. This prevents confusion and makes comparisons much clearer.

5 Research Habits for CS & AI

5.1 Daily Practices

5.1.1 Code and Experiment Management

- **Version Control**: Commit code daily with meaningful messages
- **Documentation**: Document code as you write it, not after
- **Reproducibility**: Always set random seeds and log all parameters

- **Backup**: Use git and cloud storage; never trust a single copy

5.1.2 Experimental Reproducibility: The Foundation of Credible Research

Why Reproducibility Matters

The Reproducibility Crisis: Studies show that 70% of researchers have failed to reproduce another scientist's experiments, and more than 50% have failed to reproduce their own experiments.
Your research is only as good as its reproducibility. Without it, your findings cannot be verified, extended, or built upon by others — or even by yourself six months later!

Three Pillars of Reproducible Research:

1. **Environment Management** — Exact software versions and dependencies
2. **Data and Code Versioning** — Track all changes systematically
3. **Documentation** — Clear instructions for replication

5.1.3 Environment Management with Conda

Conda is your Swiss Army knife for managing Python environments and dependencies. It ensures that your code runs identically across different machines and time periods.

Getting Started with Conda:

Essential Conda Commands

```
# Create a new environment
conda create -n myproject python=3.9

# Activate the environment
conda activate myproject

# Install packages
conda install numpy pandas scikit-learn
pip install special-package # For pip-only packages

# Export environment
conda env export > environment.yml

# Recreate environment from file
conda env create -f environment.yml

# List all environments
conda env list

# Remove an environment
conda env remove -n myproject
```

Best Practices for Conda:

- **One Project, One Environment**: Never use the base environment for projects
- **Version Pinning**: Always specify exact versions in production

- **Regular Exports**: Update environment.yml with every dependency change
- **Documentation**: Include setup instructions in your README

Pro Tip: Conda vs pip

When to use Conda:

- Scientific packages (numpy, scipy, pandas)
- Complex dependencies (CUDA, MKL)
- Cross-platform compatibility needed

When to use pip:

- Latest deep learning frameworks
- Niche or cutting-edge packages
- Package not available in conda channels

Golden Rule: Install with conda first, use pip as fallback. Always use pip *inside* a conda environment, never globally!

5.1.4 Containerization with Docker

Docker takes reproducibility to the next level by packaging your entire computational environment — OS, system libraries, and all dependencies — into a portable container.

Docker Fundamentals:

Docker Architecture

Component	Purpose
Image	Blueprint for creating containers (like a snapshot)
Container	Running instance of an image
Dockerfile	Recipe for building an image
Registry	Storage for sharing images (e.g., Docker Hub)

Sample Dockerfile for ML Research:

```
# Start from official Python image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    build-essential \
    git \
    && rm -rf /var/lib/apt/lists/*

# Copy requirements file
COPY requirements.txt .

# Install Python dependencies
```

```
RUN pip install --no-cache-dir -r requirements.txt

# Copy project code
COPY . .

# Set default command
CMD ["python", "main.py"]
```

Listing 1: Dockerfile

Essential Docker Commands:

```
# Build an image
docker build -t myproject:v1 .

# Run a container
docker run -it myproject:v1

# Mount local directory
docker run -v $(pwd):/app myproject:v1

# List containers
docker ps -a

# Push to registry
docker push username/myproject:v1
```

When to Use What?

Use Conda when:

- Working locally or on shared servers
- Need quick iteration and debugging
- Managing Python/R environments
- Team uses similar OS (e.g., all Linux)

Use Docker when:

- Deploying to production
- Sharing with external collaborators
- Need exact OS-level reproducibility
- Running on different platforms (Mac/Linux/Windows)

Use Both when:

- Maximum reproducibility is critical
- Publishing computational research
- Building ML pipelines

Reproducibility Checklist:

Before Publishing Your Research

- ☐ **Environment file** (environment.yml or requirements.txt) included
- ☐ **Random seeds** set for all stochastic processes
- ☐ **Data availability** statement with download links or instructions
- ☐ **README** with step-by-step reproduction instructions
- ☐ **Version information** for all major dependencies
- ☐ **Hardware specifications** documented (GPU, RAM, etc.)
- ☐ **Expected outputs** or results included for verification
- ☐ **Dockerfile** provided for complex environments
- ☐ **Jupyter notebooks** cleared and run top-to-bottom
- ☐ **Tests** to verify key functions work correctly

5.1.5 Working with Remote Servers and HPC Clusters

As a researcher, you'll spend significant time working on remote servers and High-Performance Computing (HPC) clusters. Mastering these environments is crucial for running large-scale experiments and accessing computational resources beyond your local machine.

Why Remote Servers Matter

Your laptop is not enough. Modern research requires:

- **GPUs**: Training deep learning models
- **Memory**: Processing large datasets (100GB+)
- **Parallel Computing**: Running multiple experiments simultaneously
- **Collaboration**: Shared environments with your research group

SSH: Your Gateway to Remote Computing

SSH (Secure Shell) is your primary tool for accessing remote servers:

```
# Basic SSH connection
ssh username@server.example.edu

# SSH with specific port
ssh -p 2222 username@server.example.edu

# SSH with key authentication (recommended)
ssh -i ~/.ssh/id_rsa username@server.example.edu

# SSH config file (~/.ssh/config) for shortcuts
Host myserver
    HostName server.example.edu
    User username
    Port 22
    IdentityFile ~/.ssh/id_rsa
```

```
# Then simply connect with:
ssh myserver
```

Pro Tip: SSH Key Setup

```
# Generate SSH key pair
ssh-keygen -t ed25519 -C "your_email@example.com"

# Copy public key to server
ssh-copy-id username@server.example.edu

# Now you can login without password!
```

Essential Server Commands

Command	Purpose
`htop`	Interactive process viewer (CPU/Memory usage)
`nvidia-smi`	GPU status and usage
`df -h`	Disk space usage
`du -sh *`	Directory sizes
`ps aux | grep username`	Your running processes
`kill -9 PID`	Force terminate a process
`nohup command &`	Run command that survives logout
`screen / tmux`	Persistent terminal sessions

Tmux: Your Best Friend on Servers

Tmux allows you to maintain persistent sessions that survive disconnections:

```
# Start new tmux session
tmux new -s experiment

# Detach from session (Ctrl+b, then d)
# Your processes keep running!

# List sessions
tmux ls

# Reattach to session
tmux attach -t experiment

# Kill session
tmux kill-session -t experiment

# Essential tmux shortcuts (after Ctrl+b):
# c - create new window
# n/p - next/previous window
# % - split pane vertically
# " - split pane horizontally
# arrow keys - navigate panes
```

Data Transfer Methods

Transferring Files to/from Servers

```
# SCP: Simple file copy
scp local_file.txt username@server:~/remote_path/
scp username@server:~/remote_file.txt ./local_path/

# Rsync: Efficient sync (resume support, compression)
rsync -avz --progress local_dir/ username@server:~/remote_dir/
rsync -avz --exclude='*.pyc' source/ dest/

# SFTP: Interactive file transfer
sftp username@server
> put local_file.txt
> get remote_file.txt
> exit

# For large datasets, use compression
tar -czf data.tar.gz data_folder/
scp data.tar.gz username@server:~/
# On server:
tar -xzf data.tar.gz
```

SLURM: Job Scheduling System

Most HPC clusters use SLURM for resource management:

```
# Submit a job
sbatch job_script.sh

# Check job status
squeue -u username

# Cancel a job
scancel job_id

# Interactive session with GPU
srun --gres=gpu:1 --mem=32G --time=4:00:00 --pty bash

# Check available resources
sinfo
```

Sample SLURM Job Script:

Listing 2: job_script.sh

```
#!/bin/bash
#SBATCH --job-name=my_experiment
#SBATCH --output=logs/%j.out
#SBATCH --error=logs/%j.err
#SBATCH --time=24:00:00
#SBATCH --mem=64G
#SBATCH --gres=gpu:v100:1
#SBATCH --cpus-per-task=8

# Load modules
module load python/3.9
module load cuda/11.7

# Activate conda environment
```

```
conda activate myenv

# Run experiment
python train.py --config config.yaml
```

GPU Management

```
# Check GPU availability
nvidia-smi

# Monitor GPU usage in real-time
watch -n 1 nvidia-smi

# Specify GPU for your program
CUDA_VISIBLE_DEVICES=0,1 python train.py

# Check CUDA version
nvcc --version

# Python: Check GPU in code
import torch
print(torch.cuda.is_available())
print(torch.cuda.device_count())
```

Server Etiquette

DO:

- Check resource availability before running jobs
- Use job schedulers for long-running tasks
- Clean up temporary files regularly
- Document your environment setup
- Communicate with team about resource usage

DON'T:

- Run intensive jobs on login nodes
- Monopolize all GPUs without coordination
- Store large datasets in home directory (use scratch/data folders)
- Leave zombie processes running
- Hardcode absolute paths in shared code

Server Workflow Best Practices:

Recommended Workflow

1. **Development**: Write and test code locally or in Jupyter
2. **Small-scale Testing**: Test on server with small dataset
3. **Resource Estimation**: Profile memory and time requirements
4. **Full Experiments**: Submit batch jobs with appropriate resources
5. **Monitoring**: Use tmux to monitor long-running jobs
6. **Results Transfer**: Sync results back to local machine
7. **Cleanup**: Remove temporary files and failed experiments

Quick Server Checklist:

☐ SSH keys configured for passwordless login

☐ Tmux/Screen session for persistent work

☐ Conda environment set up and activated

☐ Data transferred to appropriate directory

☐ Job script tested with small dataset

☐ Resource requirements estimated

☐ Output directory created for logs

☐ Backup of important results configured

5.1.6 Time Management

- **Deep Work Blocks**: Reserve 2-4 hour blocks for focused research
- **Experimentation**: Run experiments overnight/weekends when possible
- **Writing**: Write a little every day, even just notes

5.2 Research Methodology

5.2.1 Literature Review

- **Systematic Search**: Use Google Scholar, arXiv, and conference proceedings
- **Latest Papers Hub**: Hugging Face Papers - Excellent source for discovering the latest high-quality ML papers with community discussions
- **Quality Assessment**: When selecting papers, always note:
  - **Institution**: Which university or research lab produced the work
  - **Publication Year**: How recent is the research (prioritize recent work for current trends)
  - **Venue**: Where was it published (top-tier conferences/journals have rigorous peer review)

  - **Impact**: Citation count and influence in the field

- **Prioritize High-Quality Sources**:
  - Top conferences: NeurIPS, ICML, ICLR, ACL, CVPR, AAAI, IJCAI
  - Top journals: Nature, Science, JMLR, IEEE TPAMI, TACL
  - Reputable institutions with strong track records in your area
- **Note-Taking**: Summarize key ideas, methods, and limitations
- **Stay Current**: Follow top conferences (NeurIPS, ICML, ACL, CVPR, etc.)

5.2.2 Research Idea Generation

Ideas Come From Connections

Good research ideas emerge from systematic exploration, critical reading, and most importantly, discussions with senior students and peers.

I. Key Sources for Research Ideas

Source	Strategy
Senior Students	• Know what actually works vs. what sounds good • Share unpublished work and current trends • Reveal advisor expectations and preferences
Literature Gaps	• Mine "Future Work" sections from top papers • Question every "we assume" statement • Analyze where current methods fail
Cross-Pollination	• Apply technique X to domain Y • Attend talks outside your immediate area • Read papers from adjacent fields
Group Meetings	• "What do you wish you had worked on instead?" • "What tools/datasets do you wish existed?" • "What's everyone working on but shouldn't be?"

II. Evaluating Ideas: The FIVE+C Framework

Criteria	Key Questions
Feasible	Can you actually do it with your resources?
Interesting	Will the community care about this?
Novel	Is it sufficiently different from existing work?
Valuable	Does it advance the field meaningfully?
Expertise-aligned	Does it leverage your strengths?
+Collaborative	Can others contribute? Will it build relationships?

Red Flags: Too incremental • Too ambitious (needs 10 PhDs) • Already solved • No evaluation metric • Requires unavailable resources

III. The Idea Development Pipeline

- **Collection** (continuous): Daily idea journal + Weekly peer discussions + Monthly advisor brainstorming

- **Filtering** (weekly): Review ideas → Get peer feedback → Check arXiv → Rate on FIVE+C
- **Validation** (2-week sprints): Build prototype → Show to senior students → Refine → Present if promising

Build Your "Idea Exchange" Network: Meet weekly with 3-4 peers, everyone brings one idea/problem, no initial criticism—only building, document everything.

Key Lessons

- **Pattern Recognition**: If 3+ people mention the same problem, investigate it
- **Execution > Ideas**: Don't fear "theft"—implementation matters more
- **Start Imperfect**: Begin with "good enough" and iterate
- **Trust Warnings**: When seniors say "avoid this," listen carefully

Quick Validation Checklist: ☐ Discussed with 3+ people ☐ One-sentence explanation ready ☐ Checked last 2 years of conferences ☐ Have necessary resources ☐ Can prototype in 2 weeks

5.2.3 Experiment Design

- **Start Simple**: Begin with baseline implementations
- **One Variable**: Change one thing at a time
- **Hypothesis-Driven**: Always have a clear hypothesis before running
- **Negative Results**: Document what doesn't work—it's valuable!

5.3 Collaboration Best Practices

5.3.1 Communication

- **Regular Updates**: Weekly progress reports to advisor
- **Ask Questions Early**: Don't struggle alone for days
- **Share Drafts**: Get feedback on writing early and often
- **Meeting Prep**: Come with agenda and specific questions

5.3.2 Code Collaboration

- **Code Reviews**: Request reviews for significant changes
- **Documentation**: Write README files for every project
- **Modular Design**: Write reusable, testable code
- **Shared Standards**: Follow team coding conventions

5.3.3 Active Participation in Group Meetings

Why Active Participation Matters

Asking questions in group meetings is not just encouraged—it's essential for your growth as a researcher. Here's why:

Benefits for You:

- **Deeper Understanding**: Questions help clarify concepts you might have misunderstood
- **Build Connections**: Your questions can spark collaborations and discussions
- **Develop Critical Thinking**: Learning to ask good questions is a crucial research skill
- **Show Engagement**: Active participation demonstrates your interest and commitment
- **Learn from Others**: Often, your peers have the same questions but hesitate to ask
- **Practice Communication**: Formulating clear questions improves your ability to communicate research ideas

Common Types of Questions to Ask:

5.4 Professional Development

5.4.1 Skills to Develop

- **Programming**: Python, PyTorch/TensorFlow, Git
- **Mathematics**: Linear algebra, probability, optimization
- **Writing**: Technical writing, LaTeX
- **Presentation**: Public speaking, visualization

6 Report Writing Standards

6.1 Structure

Follow the standard scientific paper structure:

1. Abstract (150-250 words)
2. Introduction
3. Related Work
4. Methodology
5. Experiments
6. Results and Discussion
7. Conclusion
8. References

Question Type	Example Questions
Clarification Questions	• "Could you explain what you mean by [technical term]?" • "How does this differ from [related concept]?" • "Can you walk through that derivation/algorithm again?"
Methodology Questions	• "Why did you choose this particular approach?" • "Have you considered [alternative method]?" • "How did you handle [specific challenge]?" • "What are the computational requirements?"
Results and Interpretation	• "What's the statistical significance of these results?" • "How do these results compare to the baseline?" • "What do you think explains this unexpected outcome?" • "Have you tested this on [different dataset/scenario]?"
Broader Impact Questions	• "How does this relate to [other work in the field]?" • "What are the practical applications of this research?" • "What are the limitations of this approach?" • "What ethical considerations should we keep in mind?"
Future Work Questions	• "What are the next steps for this project?" • "How could this be extended to [related problem]?" • "What would you do differently if starting over?" • "What's the biggest challenge you foresee?"

Table 1: Common types of questions to ask during group meetings

6.2 Academic Writing Style

Some Personal Writing Advice

1. Clarity Over Complexity: Big words don't equal better writing. Choose simple, precise language over unnecessarily complex vocabulary.
2. Introduce Abbreviations Smoothly: Always define terms BEFORE abbreviations, and integrate them naturally into your text.
Bad: "...transforms raw OMOP (Observational Medical Outcomes Partnership) CDM (Common Data Model) data..."
Good: "...transforms the CRITICAL data recorded in the Observational Medical Outcomes Partnership Common Data Model (OMOP CDM)..."
Remember: Your goal is to communicate ideas clearly, not to impress with vocabulary. If a simpler word works, use it.

6.3 Common Mistakes to Avoid

Don't Do This!

1. **Wall of Text**: Never fill slides with paragraphs
2. **Unclear Axes**: Always label axes before discussing graphs
3. **Missing Context**: Never jump to results without setup
4. **Unreadable Fonts**: Test visibility from the back of the room
5. **No Rehearsal**: Always practice your timing
6. **Ignoring Questions**: Listen carefully and answer what was asked

7 Weekly Update Format

7.1 Structure

Your weekly updates should include:

1. **Progress**: What you accomplished
2. **Challenges**: What difficulties you encountered
3. **Plans**: What you'll do next week
4. **Questions**: What help you need

7.2 Best Practices

- Keep updates concise (5-10 slides)
- Include visual evidence of progress
- Be honest about challenges
- Come prepared with specific questions

7.3 Effective Communication with Your Advisor

The Foundation of Productive Meetings

Every meeting with your advisor should be a **strategic conversation**, not just a status report. Effective communication means clearly articulating problems, progress, and plans—while being ready to defend your ideas and receive guidance.

Essential Components of Effective Advisor Meetings:

1. **Define the Problem Clearly**
   - What specific research question or technical challenge are you addressing?
   - Why is this problem important to your project?
   - What context does your advisor need to understand the issue?
2. **Report What You've Done**

- What experiments did you run during this period?
- What approaches did you try?
- Show concrete results (plots, tables, code snippets)

3. **Clarify What You've Solved**
   - Which problems did you successfully resolve?
   - What insights did you gain?
   - What works and what doesn't?
4. **Identify What Remains Unsolved**
   - What challenges are you still facing?
   - Where are you stuck and why?
   - What have you tried that didn't work?
5. **Present Your Next Steps**
   - What is your concrete plan moving forward?
   - How do you intend to address the remaining challenges?
   - What timeline are you working with?
6. **Engage in Active Discussion**
   - **Questions to discuss**: What specific issues do you want to brainstorm together?
   - **Guidance you seek**: What strategic advice or direction do you need?
   - **Defend your ideas**: Be prepared to explain your reasoning and justify your approach
   - **Listen actively**: Take notes on your advisor's suggestions and ask clarifying questions

Common Pitfalls to Avoid

- **Vague updates**: "I'm still working on it" tells your advisor nothing useful
- **Hiding problems**: Being afraid to admit you're stuck wastes everyone's time
- **Passive listening**: Meetings shouldn't be one-way lectures—engage actively
- **No preparation**: Coming without a clear agenda signals lack of seriousness
- **Defensiveness**: Being unable to defend your ideas *or* being closed to feedback

Meeting Preparation Checklist

Before each advisor meeting, prepare:

- Written summary of progress (even if brief)
- Visual evidence (plots, tables, architecture diagrams)
- Specific questions ranked by priority
- Your proposed solutions to current challenges
- Concrete plan for the next period

Remember: Your advisor's time is valuable, but so is yours. Effective communication ensures both parties get maximum value from each meeting, accelerating your research progress and strengthening your advisor-student relationship.

8 How to be a Productive PhD Student

8.1 Strategic Research Framework

The Foundation of PhD Success

Becoming a productive PhD student requires more than just hard work—it demands strategic thinking, systematic organization, and deliberate practice. This section synthesizes best practices from successful researchers worldwide.

8.1.1 Clear Goals and Structured Output Paths

Define Your North Star:

- **Long-term Vision**: What impact do you want to make in your field?
- **3-Year Plan**: Break down your PhD into yearly milestones
- **Quarterly Goals**: Concrete deliverables (papers, code, experiments)
- **Weekly Targets**: Specific tasks that move you toward quarterly goals

Output-Driven Approach:

1. **Papers**: Target 2-3 quality publications per year
2. **Code**: Build reusable research tools and frameworks
3. **Presentations**: Regular practice at lab meetings and conferences
4. **Collaborations**: Actively engage in 1-2 collaborative projects

8.1.2 Daily Action System

The PhD Daily Routine

Morning (Deep Work Block):

- 2-4 hours of uninterrupted research/coding
- No email, no meetings, no distractions
- Focus on your most important task

Afternoon (Collaborative Work):

- Meetings, discussions, code reviews
- Email and administrative tasks
- Reading papers and staying current

Evening (Reflection and Planning):

- Document daily progress in research log
- Plan next day's priorities
- Light reading or skill development

8.1.3 Personal Reflection — Navigating Mode Switching in Research Life

Personal Reflection — Navigating Mode Switching in Research Life

In my own journey transitioning from a passive contributor to an independent researcher, I realized that productivity isn't only about "working harder." For me, the challenge has increasingly become switching efficiently between three cognitive modes:

- **Deep production**: when I'm writing or implementing something technically intensive;
- **Wide reading**: skimming papers, noting trends, digesting unfamiliar ideas;
- **Collaborative engagement**: managing projects, discussing ideas, attending meetings.

Each of these requires a different mindset—and I'm still building the muscle of switching between them without mental friction.
Recently, I've been trying to build small habits to ease these transitions. For example, I schedule reading during low-energy afternoons, and cluster deep work in the mornings. I'm also learning to respect the cost of context switching—it's real. And that's okay.
Progress in a PhD isn't just about depth—it's also about switching between roles with grace and clarity.

Notion-Based Management System:

- **Research Dashboard**: Track all projects, deadlines, and progress
- **Literature Database**: Organize papers with tags, notes, and connections
- **Experiment Log**: Document all experiments with parameters and results
- **Idea Capture**: Quick notes for research ideas and insights
- **Weekly Reviews**: Reflect on progress and adjust plans

8.2 Top Conferences by Field

8.2.1 AI/Machine Learning Conferences

Conference	Full Name	Focus Area	Website
Core Machine Learning (Tier 1)			
NeurIPS	Neural Information Processing Systems	General ML	neurips.cc
ICML	International Conference on Machine Learning	ML Theory	icml.cc
ICLR	International Conference on Learning Representations	Deep Learning	iclr.cc
Computer Vision			
CVPR	Computer Vision and Pattern Recognition	Vision	cvpr.thecvf.com
ICCV	International Conference on Computer Vision	Vision	thecvf.com
ECCV	European Conference on Computer Vision	Vision	eccv.ecva.net
Natural Language Processing			
ACL	Association for Computational Linguistics	NLP	aclanthology.org
EMNLP	Empirical Methods in NLP	NLP	emnlp.org
NAACL	North American Chapter of ACL	NLP	naacl.org
General AI			
AAAI	Association for Advancement of AI	General AI	aaai.org
IJCAI	International Joint Conference on AI	General AI	ijcai.org
Data Mining and Web			
KDD	Knowledge Discovery and Data Mining	Data Mining	kdd.org
WWW	The Web Conference	Web/IR	thewebconf.org

Table 2: Top-tier AI/ML conferences with submission deadlines typically 4-6 months before the conference

8.2.2 Computer Science Conferences

Conference	Full Name	Area	Website
Databases			
SIGMOD	Special Interest Group on Management of Data	Databases	sigmod.org
VLDB	Very Large Data Bases	Databases	vldb.org
Systems			
OSDI	Operating Systems Design and Implementation	Systems	usenix.org
SOSP	Symposium on Operating Systems Principles	Systems	sosp.org
Graphics			
SIGGRAPH	Special Interest Group on Graphics	Graphics	siggraph.org
Human-Computer Interaction			
CHI	Conference on Human Factors in Computing	HCI	chi.acm.org
UIST	User Interface Software and Technology	HCI	uist.acm.org
Security			
S&P	IEEE Symposium on Security and Privacy	Security	ieee-security.org
CCS	Computer and Communications Security	Security	sigsac.org
USENIX	USENIX Security Symposium	Security	usenix.org
Programming Languages			
PLDI	Programming Language Design and Implementation	PL	pldi.sigplan.org
POPL	Principles of Programming Languages	PL	popl.mpi-sws.org

Table 3: Top-tier Computer Science conferences across different research areas

8.2.3 AI for Science Conferences and Venues

The Rising Field of AI4Science

AI for Science is an emerging interdisciplinary field where machine learning meets traditional sciences. These venues offer unique opportunities to publish work that bridges AI and scientific domains.

Venue	Type	Website/Info
Workshops at Major ML Conferences		
AI4Science @ NeurIPS	Workshop	ai4sciencecommunity.github.io
AI4Science @ ICLR	Workshop	ai4sciencecommunity.github.io
AI4Science @ ICML	Workshop	ai4sciencecommunity.github.io
ML4Physical Sciences @ NeurIPS	Workshop	Annual at NeurIPS
Computational Biology @ ICML	Workshop	Annual at ICML
Journals		
Nature Machine Intelligence	Journal	nature.com/natmachintell
Science Robotics	Journal	science.org
Digital Discovery	Journal	RSC Publishing
npj Computational Materials	Journal	nature.com
Physical Review X	Journal	aps.org
Specialized Conferences		
MLCB	Machine Learning in Comp Bio	mlsb.io
ISMB/ECCB	Bioinformatics	iscb.org
AIChE	Chemical Engineering + AI	aiche.org

Table 4: AI for Science publication venues combining machine learning with scientific disciplines

8.3 Publication Strategy

8.3.1 Conference Timeline Management

Critical: Plan Your Submissions 6 Months Ahead

Most top conferences have deadlines 5-6 months before the conference date. Missing a deadline means waiting a full year for the next opportunity!
Typical Timeline for a Paper Submission:

- **T-6 months**: Start experiments and writing
- **T-3 months**: Complete main experiments
- **T-1 month**: Finish first draft, get feedback
- **T-1 week**: Polish, proofread, prepare supplementary
- **T-0**: Submit and celebrate!

Pro Tip: Set Your Personal Deadline 1 Week Early

By finishing your paper or project one week ahead of the actual deadline, you gain not only buffer time for polishing, but also a powerful social advantage: When others are in panic mode during the final week, you can *offer timely help*. This creates a natural opportunity to:

- Strengthen relationships with your peers
- Learn how others structure their research and writing
- Discover collaboration potential for future work

Many research partnerships start with a helping hand in a high-stress moment. Your preparation becomes an opportunity to build your academic network while genuinely supporting your community.

Code Anonymization for Double-Blind Review

Essential Tool: anonymous.4open.science — Creates anonymous versions of your GitHub repository for paper submissions. It removes all author information and commit history while providing a stable URL for reviewers. Simply enter your GitHub URL, and it generates an anonymized copy instantly.

8.3.2 Quality vs. Quantity Balance

The Publication Pyramid:

1. **Flagship Paper** (1 per year): Your best work targeting top venues (NeurIPS, ICML, CVPR)
2. **Solid Contributions** (1-2 per year): Good work at specialized conferences
3. **Workshop Papers** (2-3 per year): Early-stage ideas, work-in-progress

My Personal Reflection: Quality Over Quantity

Through extensive discussions with senior researchers and advisors, I've learned an invaluable lesson about publication strategy. Many successful professors have shared that while they initially pursued quantity—publishing at every workshop and conference—they ultimately realized a crucial truth:
Your career will ultimately be defined by your best work, not your longest publication list.
Yes, early in your PhD, it's perfectly fine—even advisable—to publish "quantity-type" papers. They help you:

- Learn the review process and improve your writing
- Build confidence and momentum
- Establish early credibility in your field

But to truly stand out in your field, to secure competitive faculty positions or top industry jobs, you need **representative work**—papers that people remember, cite, and build upon. These are the papers that:

- Introduce novel ideas or significant improvements
- Are thoroughly validated with extensive experiments
- Tell a compelling story that resonates with the community
- Become part of your "research identity"

My advice: Start with quantity to build skills and confidence, but always be working toward that signature contribution. When you're ready, invest 6-12 months in a project that could define your research trajectory.

8.3.3 Collaboration Guidelines

Authorship Best Practices:

- **First Author**: Lead the project, write majority of paper
- **Contributing Author**: Significant experiments or sections
- **Advisory Author**: Guidance and feedback
- Always discuss authorship order early in the project
- Document contributions for each paper

8.3.4 Technical Paper Planning: Key Points to Think Through

Before You Start Writing a Technical Paper

A successful technical paper isn't just about good results—it's about clear thinking. Before writing a single word, ensure you can answer these strategic questions about your work.

I. Paper Core Elements
1. Main Contribution:

- What is my core innovation? Is it theoretical innovation, algorithm design, system optimization, application scenario, or analysis/unification of existing methods?
- Can I explain in one sentence: "We are the first to solve X important problem"?
- Do I have multi-level contributions? (theory + practice; new method + new task; new analysis + new benchmark)

2. Results Presentation Strategy:

- Which experiments are **must-have** core comparisons?
- Which are **supporting** ablation studies / case studies / visualizations?
- Do I need multiple benchmarks (across vision/NLP) to show generality?
- Can I achieve SOTA with fewer resources or higher efficiency, or significantly outperform on specific data/tasks?

II. Pre-Writing Strategic Questions
1. Topic Positioning:

- Is my topic hot at this conference this year? Is it oversaturated?
- Does it align with emerging trends? (e.g., "Multimodal LLM", "efficient finetuning", "long context modeling", "AI4Science")
- Does it match past hot sessions/workshops/invited talks at the venue?

2. Innovation Assessment:

- How is my work clearly different from the past year's related work? (not just a simple tweak)
- If it's combinatorial innovation (A+B), is it natural, effective, and insightful?
- Do I have a unique perspective or counter-intuitive but effective discovery?

3. Method vs Insight Framing:

- Is my method just a "reasonable trick" or does it reveal a new phenomenon/structure/principle?
- Which framing is better: a new model? a new phenomenon? a more general framework?
- Is it worth abstracting as "xxx is all you need", "rethinking xxx", or "beyond xxx"?

4. Dataset and Task Design:

- Am I proposing a representative new benchmark or task?
- If using existing data, have I chosen the right entry point/task definition/evaluation metrics?
- Have I proposed a compelling motivation from the application perspective? (e.g., "current models still fail in scenario X")

5. Reproducibility and Generalizability:

- Can others easily reproduce my work? (crucial for NeurIPS/ICLR)

- Can the method generalize to other domains?
- Have I done sufficient general analysis or interpretability experiments to support my claims?

III. Strategic and Risk Considerations
1. Maturity vs First-Mover:

- Is my work better suited for "careful polishing and mature submission" or "submit first round, iterate quickly"?
- Should I post on arXiv first to establish timeline, or wait for a complete version?

2. Submission Roadmap:

- Should I use workshop/spotlight/poster/oral strategy to familiarize with the field community first?
- If rejected this round, what's my backup venue? (ICML → NeurIPS → ICLR → domain-specific conferences)
- How does this paper fit into my overall PhD publication strategy?

Quick Checklist Before Starting:

☐ Can I explain my contribution in one sentence?

☐ Do I have a clear experimental validation plan?

☐ Have I checked recent papers (last 6 months) to ensure novelty?

☐ Is my framing aligned with conference trends?

☐ Do I have a backup plan if rejected?

8.3.5 Writing Effective Survey/Review Papers

The Value of Survey Papers

A truly good survey is more than just a literature dump. It requires simultaneous mastery of information density, structural clarity, and research insights.

I. Core Skills for Survey Papers
1. Fast Retrieval & Organization:

- Provide well-organized literature maps for the research community
- Enable readers to quickly find papers related to their specific tasks/problems/paradigms
- Create clear categorization, structured tables, and systematic summaries
- **Goal**: Readers can locate relevant work just from your figures/tables without reading original papers

2. Insight & Guidance:

- Help readers understand the entire research direction's structure, gaps, evolution, and future opportunities
- Address key questions:

  - Which directions are over-studied vs. neglected?
  - Which methods/techniques are saturated vs. immature but promising?
  - What are the key challenges/controversies/unsolved problems in the field?

II. Providing Organizational Frameworks
3. Structured Thinking Approaches:

- Don't just list "which papers exist" but teach "how to organize this research domain"
- Develop clear organizational frameworks:
  - **Clinical Pathway Framework** for medical AI papers
  - **Task vs Input-Output Type Matrix** for NLP surveys
  - **Backbone vs Head Layer Framework** for Vision domains
  - Clear architectural distinctions (e.g., LLM vs Agent systems)

4. Building Knowledge Taxonomy:

- Help clarify ambiguous or misleading terms, paradigms, and task definitions
- Critical clarifications to address:
  - What are the essential differences between LLM and Agent systems?
  - Are Diagnosis tasks and Reasoning tasks really evaluated the same way?
  - Are some SOTA results just "prompting tweaks" rather than fundamental innovations?
- This standardization is especially important in fast-moving domains (e.g., medical LLMs)

III. Critical Perspective
5. Beyond Summarization:

- Don't just summarize existing work—identify "what's problematic" and "which assumptions deserve rethinking"
- Example critical insights:
  - "Most models assume perfect structured inputs, which rarely exist in real clinical settings."
  - "Current benchmarks fail to reflect the coordination burden that dominates real-world workflows."
  - "The field conflates correlation with causation in many evaluation metrics."
- Such "reflective surveys" are considered more insightful and inspire future research

Common Pitfalls in Survey Papers

Avoid these mistakes:

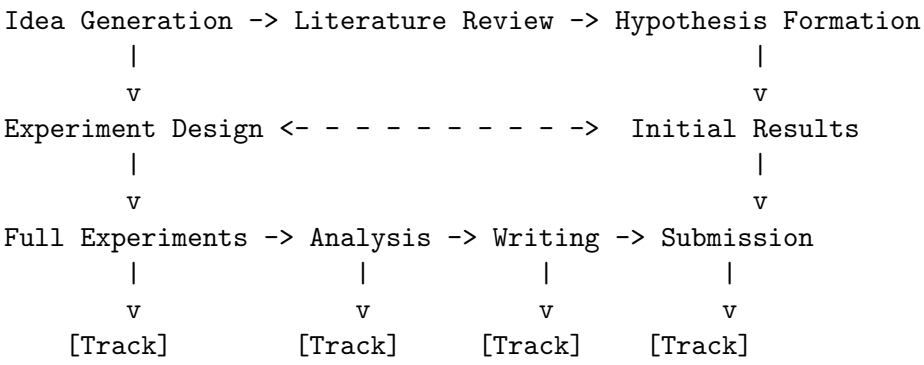
- **Literature dumping**: Just listing papers without synthesis
- **Lack of critical analysis**: Only describing without evaluating
- **Poor organization**: No clear framework or taxonomy
- **Missing recent work**: Surveys become outdated quickly in fast-moving fields
- **Narrow scope**: Focusing only on one approach without considering alternatives

Quick Checklist for Survey Papers:

- ☐ Comprehensive literature coverage with clear categorization
- ☐ Organizational framework that structures the domain
- ☐ Critical analysis of gaps and limitations
- ☐ Future research directions clearly identified
- ☐ Tables/figures that standalone convey key information
- ☐ Coverage of very recent papers (last 6-12 months)
- ☐ Clear definitions and taxonomy for the field

8.4 Productivity Systems and Tools

8.4.1 Research Pipeline Management



Tools for Each Stage:

- **Idea Generation**: Notion, Obsidian, Research diary
- **Literature Review**: Zotero, Mendeley, Connected Papers
- **Experiments**: Weights & Biases, MLflow, Sacred
- **Writing**: LaTeX + Git, Overleaf for collaboration
- **Project Management**: GitHub Projects, Trello, Asana

8.4.2 Time Management Strategies

The 80/20 Rule for PhD Productivity

Focus 80% of your time on:

- Core research that directly contributes to papers
- Writing and improving your communication skills
- Building deep expertise in your specific area

Limit to 20% of your time:

- Attending talks outside your direct area
- Administrative tasks and service
- Exploring tangentially related topics

8.4.3 AI Tools for PhD Research

The AI-Augmented Researcher

The research landscape has fundamentally shifted. AI tools are no longer optional productivity hacks—they are becoming core infrastructure for competitive research. A modern PhD student who learns to wield these tools effectively can compress months of manual work into days, and spend their cognitive energy on what actually matters: original thinking.

This section covers the two highest-leverage applications I've personally found: AI-assisted literature management and AI coding agents.

I. AI-Assisted Literature Review

Literature review is notoriously the most time-consuming yet cognitively unrewarding part of early research. The good news: it's also where AI provides the most immediate return.

Semantic Scholar API — Your Literature Intelligence Layer

Semantic Scholar is an AI-powered academic search engine with a free API. Unlike Google Scholar, it exposes structured metadata programmatically:

```
# Search papers by keyword
curl "https://api.semanticscholar.org/graph/v1/paper/search\
  ?query=causal+inference+LLM\
  &fields=title,year,authors,citationCount,abstract\
  &limit=10"

# Get citation network of a specific paper
curl "https://api.semanticscholar.org/graph/v1/paper/{paper_id}/citations\
  ?fields=title,year,authors"

# Find papers that cite a landmark paper (build on prior work)
curl "https://api.semanticscholar.org/graph/v1/paper/{paper_id}/references\
  ?fields=title,year,authors"
```

Practical workflows enabled by the API:

- **Citation snowballing**: Automatically traverse citation graphs to find all relevant work
- **Trend detection**: Track how many papers per year cite a method to gauge its lifecycle
- **Baseline hunting**: Find the actual SOTA by sorting by citation count + recency
- **BibTeX generation**: Pull structured metadata directly instead of copy-pasting from PDFs

NotebookLM as a Living Knowledge Base

Personal Experience: From 524 URLs to 269 Clean Sources in One Session

When I needed to systematically learn a new tool's entire documentation (500+ pages), I used an AI agent to automatically fetch the sitemap, bulk-import every page as an individual source into NotebookLM, deduplicate the translated versions, and run an audit. The result: a clean, searchable knowledge base with zero duplicates—built while I was doing other work.

The same pattern works for any domain: bulk-import relevant papers, have the system clean and organize them, then query the resulting knowledge base with precise questions instead of re-reading papers from scratch.

Google NotebookLM is an AI knowledge engine built on Gemini. Its core strength is **source-grounded responses**—every answer cites the exact source passage, so you can verify and trace claims back to the original paper. Unlike general LLMs, it doesn't hallucinate references because it only draws from documents you've explicitly provided.

High-value use cases for researchers:

Use Case	How to Execute
Paper-set Q&A	Import 20–50 papers on a topic, ask cross-paper synthesis questions
Related work drafting	Import your candidate citations, prompt: "Summarize how each paper relates to [your method]"
Methodology comparison	Import baselines, ask: "Compare the evaluation protocols across these papers"
Gap identification	Ask: "What problems do these papers explicitly leave for future work?"
Podcast generation	Auto-generate a two-person dialogue summarizing your paper set—useful for on-the-go review

Key limitation to know: NotebookLM has no public API, so bulk operations (importing 100+ URLs, deduplication, auditing) require browser automation. This is where AI agents become powerful as an orchestration layer—see the next section.

II. AI Coding Agents

Modern AI coding agents are not autocomplete tools. They can understand your codebase, reason about architecture decisions, write and run tests, and iterate on bugs autonomously. The productivity difference between researchers who use them well and those who don't is already significant—and widening.

What AI coding agents can do for your research:

- **Boilerplate elimination**: Dataset loading, training loops, evaluation scripts, logging setup—these are solved problems. Describe what you need and get working code in seconds.
- **Debugging**: Paste an error and the surrounding context; the agent traces the root cause and proposes a fix. Especially useful for cryptic CUDA errors and shape mismatches.
- **Refactoring**: "Make this reproducible," "Add argument parsing," "Wrap this in a proper experiment config"—structural improvements without manual tedium.
- **Rapid baseline implementation**: Implementing a new baseline from a paper? Give the agent the relevant paper section and ask it to implement the method. Use it as a starting point, then verify carefully.

Critical: Verify AI-Generated Code

AI coding agents make mistakes—especially on novel architectures, edge cases, and domain-specific conventions. Never submit AI-generated code to a paper without:

1. Running it and verifying outputs match expected behavior
2. Reading it line by line to understand what it's doing
3. Testing edge cases (empty batches, single-sample inputs, extreme values)
4. Having a labmate review it if it's central to your results

The goal is to use AI to eliminate tedium, not to outsource thinking. Your name is on the paper.

Recommended tools and when to use them:

Tool	Best For	Notes
Claude Code	Long-horizon tasks: refactoring entire codebases, writing complete pipelines, multi-file changes	Runs in your terminal; can read/write files directly
GitHub Copilot	Inline completion during active coding	Integrated into VS Code/JetBrains
Cursor	Mixed mode: inline suggestions + chat for local files	Strong for navigating unfamiliar codebases
Gemini in Colab	Notebook-level tasks; good for data exploration	Free tier available; native Google Drive integration

III. AI-Augmented Research Workflow

The real leverage comes from combining these tools into a unified pipeline:

```
Topic Identified
      |
      v
Semantic Scholar API  -->  Candidate paper list (structured metadata)
      |
      v
NotebookLM Notebook   -->  Bulk import, deduplicate, audit
      |
      v
AI Q&A Sessions       -->  Synthesize related work, identify gaps
      |
      v
AI Coding Agent       -->  Implement baseline, set up experiments
      |
      v
You                   -->  Original insight, critical judgment, writing
```

This pipeline doesn't replace your thinking—it removes the friction before and after your thinking, so more of your time goes to the parts only you can do.

Personal Reflection: Redefining What "Doing Research" Means

When I started my PhD, I spent the first months reading papers one-by-one, manually extracting key ideas, and feeling perpetually behind. The mental model was: reading more = knowing more = better research.

What I've gradually learned is that the bottleneck in most research isn't information access—it's the quality of your questions and the rigor of your experiments. Once I started using AI tools to handle the information plumbing (literature organization, code scaffolding, experiment tracking), I found I was spending more time on the parts of research I actually find rewarding: thinking about why something doesn't work, finding the cleaner formulation, or noticing that two apparently separate problems are actually the same.

These tools won't make your research ideas better. But they can free up enough mental space that you have the bandwidth to make them better yourself.

8.4.4 Mental Health and Sustainability

Maintaining Long-term Productivity:

- **Regular Breaks**: Take weekends off, use vacation time
- **Physical Health**: Exercise 3-4 times per week
- **Social Connections**: Maintain relationships outside research
- **Hobbies**: Keep non-academic interests alive
- **Support Network**: Build relationships with peers and mentors
- **Professional Help**: Campus counseling is usually free for students—see Section 8.5

Dealing with Imposter Syndrome:

You're not alone—70% of PhD students experience imposter syndrome. It's especially common after conferences, reading impressive papers, or starting in a new lab.

Recognition Signs:

- "I don't belong here" or "I'm not smart enough"
- Attributing success to luck rather than ability
- Fear of being "found out" as incompetent
- Comparing yourself constantly to others

Coping Strategies:

- **Keep a "Wins Journal"**: Document achievements, positive feedback, and progress
- **Normalize the Struggle**: Everyone faces challenges—they just don't advertise them
- **Focus on Growth**: Compare yourself to your past self, not to others
- **Talk to Peers**: You'll discover everyone feels this way sometimes
- **Reframe Thoughts**:
  - "I don't know enough" → "I'm here to learn"
  - "That was just luck" → "I created the conditions for success"

– "Everyone is smarter" → "Everyone has different strengths"

Remember: PhD is a Marathon, Not a Sprint

Sustainable productivity beats intense bursts followed by burnout. Build systems that support consistent, long-term progress rather than relying on heroic efforts.

8.4.5 Understanding My Cognitive Mode Switching and Rest

Author's Note

This section captures my personal reflection during an emotionally intense moment. It's not a guideline, but rather an invitation for you to pause, reflect, and realign with your own journey.

Personal Reflection: Why I Struggled with Rest

"The truly high-quality life doesn't depend on bursts of intensity, but on long-term sustainable accumulation and creation."

I often found myself internalizing that rest = laziness. Or I unconsciously avoided rest because it confronted me with deeper fears: lack of direction, fear of inadequacy, fear of failure. Even without external accountability, I would still hear the voice of an "inner advisor" telling me I wasn't enough.

This intense guilt and anxiety when I stopped working came from a classic case of **internalized pressure and self-imposed standards**. The root causes I discovered in myself included:

1. Internalized Self-Evaluation System — my worth tied to productivity
2. Conditioned response where progress meant safety
3. Emotional avoidance when still — activity as escape

In my case, it wasn't about slacking off — it was about **being afraid of who I am without output**.

But I've learned that rest isn't absence of value — it's the space for regeneration.

Rest is not reward for good behavior. It is the prerequisite for good behavior.

Like recharging a battery — you wouldn't run a machine non-stop and expect it to work flawlessly forever.

Three Key Functions I've Discovered in Real Rest

Function 1: Energy Restoration
My brain and body are like an electric vehicle — no matter how advanced, they won't run far without recharge.

Without high-quality rest, I'm not "out of willpower" — I'm out of charge.

Function 2: Idea Incubation
My breakthrough ideas often arise when I'm **not** actively thinking — during walks, showers, or sleep. That's my **Default Mode Network (DMN)** at work.

Function 3: Life Rhythm Regulator
Rest is how I regulate long-term momentum. Instead of "push and collapse," I've chosen *rhythmic sustainability*:

- Stability enables my consistency
- Flexibility enables my resilience
- Recovery enables me to scale

The Dangerous Myth I Had to Overcome: "Rest = Regression"

I used to carry these hidden inner scripts that needed updating:

My Old Script	My Updated Belief
"I can't rest yet"	"High-quality rest = readiness for tomorrow's work"
"I'll fall behind"	"Recovery helps me go further"
"Everyone else is grinding"	"Others are sprinting; I'm training for the marathon"
"I haven't earned rest"	"I'm not a machine. Rest is strategy, not luxury."

My Realization: This isn't weakness — it's my refusal to participate in a broken output culture.

8.4.6 My Struggle with the Planning Fallacy

Author's Note

This section shares my ongoing battle with over-promising and under-delivering. It's a deeply personal reflection on one of the most common yet emotionally taxing aspects of PhD life.

In research, I constantly find myself making the same mistake: "This week I'll complete the experiments, analyze the data, and write the first draft" — only to discover by the weekend that I've barely finished half of what I promised. This pattern isn't just frustrating; it creates a cycle of guilt and anxiety that can be devastating to mental health and productivity.

Why This Happens: The Planning Fallacy

Planning Fallacy: The tendency to systematically underestimate the time needed to complete tasks, even when we've experienced similar delays before.
This cognitive bias affects virtually everyone in research because:

- We focus on ideal scenarios while ignoring potential obstacles
- We underestimate uncertainty and unexpected complications
- We overestimate our control over external factors

Combined with **optimistic bias** — our tendency to believe things will go better than they statistically should — this creates perfect conditions for chronic over-promising.

Why I Keep Falling Into This Trap:
There are three main forces that drive my over-promising habit:

External Pressures:

- Advisor expectations and weekly meeting commitments
- Collaboration deadlines with other researchers
- Conference submission deadlines
- The need to appear "productive" and "on track"

Internal Motivations:

- My genuine excitement about research progress
- Fear of disappointing mentors and collaborators
- Desire to prove my worth as a capable researcher
- Impatience with the inherently slow nature of good research

Cognitive Distortions:

- I remember successful weeks more vividly than difficult ones
- I discount the frequency of unexpected complications
- I assume past delays were "one-off" events rather than the norm

My Strategies for Change

After years of this cycle, I've developed several practical approaches that have genuinely helped:

1. Ruthless Task Decomposition: Instead of vague goals like "analyze the data this week," I break everything down to the smallest meaningful units with specific time estimates. Then I multiply each estimate by 1.5-2x as a buffer for inevitable complications and unexpected issues.

2. External Reality Checks: I keep a simple log: *Original estimate vs. Actual time spent*

Task	Estimated	Actual
Literature review	4 hours	9 hours
Code debugging	2 hours	7 hours
Writing methods	6 hours	12 hours

After tracking for a few weeks, I discovered my personal "multiplier" — I consistently underestimate by about 2.3x.

3. Defensive Scheduling:

- I only commit to what I can do in 60% of my available time
- I schedule "buffer blocks" for inevitable overruns
- I separate "optimistic timeline" (internal planning) from "committed timeline" (external promises)

My Personal Realization

The planning fallacy isn't a character flaw — it's a natural human tendency amplified by the unpredictable nature of research. Learning to work *with* this tendency, rather than fighting it, has been transformative for both my productivity and my peace of mind.

I've stopped trying to be the researcher who perfectly estimates everything. Instead, I've become the researcher who builds realistic buffers, tracks patterns, and manages expectations — including my own.

The mindset shift that changed everything: Over-promising isn't being "ambitious" — it's being unrealistic. True ambition is sustainable progress built on honest self-knowledge.

8.5 Health, Friends, and Professional Support

Why This Gets Its Own Section

A PhD removes most of the structure that kept you going before. Deadlines are far apart, feedback takes months, and nobody checks whether you slept or saw a friend this week. Many of us arrive running on an old engine of fear, comparison, and self-criticism. That engine can win exams, but over five years of slow feedback it tends to burn out. What carries you instead is unglamorous: sleep, movement, people, and asking for help early. I learned this later than I should have.

8.5.1 Your Body Is Infrastructure, Not a Reward

- **Fix your wake-up time first.** A consistent wake-up time plus 10–30 minutes of outdoor light in the morning does more to repair a drifting schedule than forcing yourself to fall asleep early.
- **No big decisions late at night or on bad days.** Decide when you are calm and rested. On a bad day, only execute what you already decided—even if you execute it poorly.
- **Move most days, for reasons you get today.** Pick something you would enjoy even with no weight or appearance goal: a gym session, a boxing class, a long walk. On low-drive evenings, exercising first often brings the drive back. If the voice behind your workout is "nobody will like me until I look different," that is the old exam engine in gym clothes.
- **Judge leisure by how you feel afterward.** Rest that leaves you recovered and ready for tomorrow is good rest. A gaming or scrolling binge that leaves you more tired and asleep at 3 a.m. is not, however good it felt in the moment.
- **Treat health admin as real work.** Checkups, prescriptions, the dentist—put them on the calendar like meetings.

The Four Foundations of a Good Working State

1. **Energy**: sleep, food, movement, physical health
2. **Direction**: a clear plan and a concrete next step
3. **Drive**: a mindset that lets you start without self-attack
4. **Environment and feedback**: a good place to work (an office or library rather than your bed) and people who see your output

I used to think a good plan plus willpower was enough. They sit on top of the other two. When you are stuck, check from the bottom up: energy first.

Minimum Protocol for Low Days

When everything feels heavy, don't ask whether you feel like it, and don't re-evaluate your whole life. Run a short preset list:

☐ Get outside into daylight for 10–30 minutes

☐ Eat one normal meal and take a shower

☐ Open your main project for 5–25 minutes and do one small, concrete action

☐ At night, write down what happened—facts only, no grading

Rules I keep: Below 60 points, no scoring—only rescue. When self-criticism shows up, make the task smaller, not bigger. Within five minutes of drifting off course, do one action that brings you back.

8.5.2 Friends Are Part of the Infrastructure

Isolation is a quiet risk in a PhD. Alone with slow feedback, your inner critic becomes your only reviewer. Friends remind you that you are a person, not a progress bar.

- **Keep one standing non-work plan every week**: a meal, a sport, a walk. Put it on the calendar so it survives busy weeks.
- **Have at least one friend you can tell the truth to.** "I've been pretty low lately" is a complete sentence; it doesn't need a solution attached.
- **Keep friends outside your lab and field.** They don't measure you in citations.
- **Work near people.** For many of us, an office, a library, or a co-working session works far better than working alone at home.
- **Don't wait until you've recovered to reach out.** Good friends and a good advisor are people you can lean on now, not a reward for getting better.

What to Tell Whom

Who	What to say
Most people (classmates, acquaintances)	Nothing, or "I've been adjusting lately." You don't owe anyone an explanation.
Close friends	The truth, briefly. It usually brings you closer rather than hurting your image.
Advisor and collaborators	Your state in one line, then the plan. Details and labels are optional.

Explanation vs. Excuse

The difference is not whether you mention a hard time; it's what follows. An **excuse** stops at the state: "I was in bad shape, so it didn't get done." An **explanation** continues to ownership: "I was in bad shape and progress slipped. I'm adjusting and getting support. Next, I'll deliver X by Y."

A message to your advisor might read:

> *"I haven't been in great shape these past few weeks, and progress has been slower than planned—sorry about that. I'm actively working on it, including getting support through campus counseling. My next step is X, and I'll show you Y by [date]."*

What rebuilds trust is not the wording but the small promises you keep afterward.

8.5.3 Talking to a Professional

Seeing a counselor or therapist is not a last resort, and it says nothing bad about you. It is like seeing a doctor when an injury doesn't heal on its own. Most universities make counseling free or low-cost for students, and starting early usually shortens the hard stretch.

Signs it's worth booking a conversation:

- Low mood, flatness, or loss of interest that lasts more than two weeks
- A sleep schedule that keeps drifting despite your efforts, or mornings that feel like the worst part of the day
- Comparing yourself with peers until you feel hopeless, or thinking "I'm just pretending so I can be lazy"
- Months of new insight about yourself with little change in what you actually do

- You've thought "maybe I should talk to someone" more than twice

Thoughts That Delay Help

The thought	What's closer to true
"I need to figure out why first."	Recovery doesn't require a full explanation. Many people understand the causes only afterward, and working them out is something you can do with a counselor.
"My advisor, friends, and environment are great. I have no reason to feel this way."	Low mood doesn't check your circumstances, any more than the flu does.
"I'm making excuses, or faking it."	People who are simply lazy rarely interrogate themselves this hard. That doubt is itself a common part of feeling low.
"I'll reach out once I'm a bit better."	This is the most common delay. Reaching out is part of getting better, not a reward for it.
"It's awkward to go back to my old therapist after disappearing."	Clinicians see returning clients all the time. "I'd like to start again" is enough.

How to start (US universities):

1. **Find your counseling center.** Services are usually covered by the student health fee. At Harvard, CAMHS offers services at no cost and starts with a 20-minute phone consultation; call (617) 495-2042 and press 1 during business hours.
2. **Say briefly what's going on.** You don't need perfect words. Bringing a few notes is fine.
3. **Ask about longer-term care if you want it.** Campus centers often work short-term and can refer you to an outside therapist who takes your student insurance.
4. **Switch if it isn't a fit.** Finding the right clinician sometimes takes two tries; that's normal.
5. **Medication questions** go to a psychiatrist or another prescriber; your counselor or primary care provider can refer you.

If You Are in Crisis

If you ever think about ending your life or hurting yourself, reach out now. In the US, call or text **988** (988 Suicide & Crisis Lifeline). Harvard students can also call the 24/7 CAMHS Cares line at **(617) 495-2042**. Outside the US, call your local emergency number. Telling a friend you trust right away counts too.

8.6 Academic Social Networking

8.6.1 Building Your Research Network

Conference Networking Strategy:

1. **Before the Conference**:
   - Review accepted papers and identify interesting authors

- Prepare your elevator pitch (30 seconds about your research)
- Schedule meetings with researchers you want to meet
- Join conference Slack/Discord if available

2. **During the Conference**:
   - Attend poster sessions—easiest place to meet people
   - Ask thoughtful questions during Q&A
   - Join social events and workshops
   - **Sit with strangers at lunch**—not just your labmates
   - Stand near coffee breaks—natural conversation starter
   - Exchange contact information and follow up
3. **After the Conference**:
   - Send follow-up emails within 48 hours (while memory is fresh)
   - Share relevant papers or resources
   - Consider collaboration opportunities
   - Stay connected via social media/email
   - Add new contacts to your network spreadsheet

Practical Conversation Starters:

- "What brings you to this conference?"
- "I really enjoyed your talk/poster on X. Have you considered Y?"
- "Which talks are you most excited about?"
- "How does your work relate to [current session topic]?"
- "I'm working on something similar—could I ask your thoughts on X?"

Networking Tips for Introverts:

- **Set small goals**: Meet 2-3 new people per day, not 20
- **Use the buddy system**: Network with a more outgoing colleague
- **Volunteer**: Being a session chair or volunteer gives you a role
- **One-on-one meetings**: Schedule coffee chats instead of group events
- **Take breaks**: Step outside to recharge between sessions

8.6.2 Online Presence

Essential Platforms for Researchers:

- **Google Scholar**: Track citations and create your profile
- **Twitter/X**: Follow researchers, share papers, join discussions
- **LinkedIn**: Professional networking and job opportunities
- **GitHub**: Share code and collaborate on projects
- **Personal Website**: Portfolio of your work and research statement

8.6.3 Building Your Research Brand Online

Strategic Platform Use

Focus on 2-3 platforms consistently:

- **LinkedIn**: Professional updates, connect with researchers, join academic groups
- **Twitter/X**: Share papers, conference insights, engage with research community
- **Handshake**: Industry opportunities and internships
- **Google Scholar**: Keep publications updated

Realistic Update Schedule

Sustainable Posting Frequency:

- **LinkedIn**: 1-2 times per month (major updates, papers, achievements)
- **Twitter/X**: 1-2 times per week (paper shares, thoughts, retweets)
- **Personal website**: Monthly updates
- **GitHub**: When releasing code/projects

Key principle: Consistency matters more than frequency. Better to post once a week reliably than daily for a month then disappear.

Content Strategy

What to Share:

- Your published papers with 1-2 sentence summary
- Interesting papers from others in your field
- Conference attendance and key takeaways
- Research milestones and achievements
- Thoughtful comments on trending research topics

The 80/20 Rule: 80% sharing others' work and insights, 20% self-promotion

Quick Start Checklist

1. Pick 2 main platforms to focus on
2. Use same professional photo everywhere
3. Write clear bio: "PhD student at [University] working on [specific topic]"
4. Follow 20-30 key researchers in your field
5. Set weekly reminder to post/engage
6. Comment before connecting - build relationships gradually

Key Points:

- **Start small**: Don't try to be everywhere at once
- **Be authentic**: Share genuine thoughts, not just announcements
- **Engage meaningfully**: Quality interactions > quantity
- **Stay consistent**: Regular modest activity beats sporadic bursts

8.6.4 Building Your Academic Website

Your Digital Academic Identity

Your personal website is often the first result when someone searches your name. It's your opportunity to control your narrative and showcase your work professionally.

Why You Need a Personal Website:

- First Google result for your name—control what people see
- Showcase work beyond just papers (code, visualizations, blog posts)
- Professional presence for job applications
- Central hub linking all your other profiles

Recommended Website Templates:
1. Static Site Generators (Free hosting on GitHub Pages):

- **Hugo Academic**
  - Most popular template for researchers
  - Built-in publication management
  - Clean, professional design
  - Active community support
- **al-folio**
  - Beautiful Jekyll theme for academics
  - Minimal, elegant design
  - Great publication and project pages

  - Used by many CS researchers

- **AcademicPages**
  - Fork of Minimal Mistakes theme
  - Simple to customize
  - Good for beginners
  - Step-by-step setup guide

- **Jekyll** with Academic Themes
  - Minimal Mistakes - Highly customizable Jekyll theme
  - Modern Resume - Clean CV/resume focused
  - Minima - Jekyll's default theme, very simple
  - Beautiful Jekyll - Ready-to-use template
  - Free hosting on GitHub Pages with custom domain support

2. For More Dynamic Sites:

- **Next.js/React-based templates**: Modern, fast, good for CS students who code
- **WordPress with Academic themes**: No coding required, easier updates
- **Notion/Carrd**: Quick setup, limited customization

Essential Pages to Include:

1. **Home/About**: Professional photo, brief bio, research interests
2. **Publications**: Papers with PDFs/links, organized by year
3. **Projects/Research**: Current and past projects with descriptions
4. **CV/Resume**: Downloadable PDF, always up-to-date
5. **Blog** (optional): Technical posts, research updates
6. **Contact**: Email, office location, social media links

Quick Setup Tips:

- Start with a template—don't build from scratch
- Use Google Scholar for managing publication citations
- Include keywords for SEO (your research area, university, etc.)
- Keep design clean and professional—content over aesthetics
- Update at least quarterly (after each paper/project)
- Test on mobile—many people browse on phones

8.7 Essential Reading for PhD Students

Must-Read Books and Articles

These readings provide invaluable insights into the PhD journey, research life, and academic career development. Each has helped thousands of students navigate their doctoral studies more effectively.

8.7.1 Core PhD Experience

- **The PhD Grind by Philip Guo**
  - A candid memoir of one student's six-year journey through a computer science PhD at Stanford
  - Provides honest insights into the struggles, failures, and eventual successes
  - Essential reading for understanding the emotional and practical realities of PhD life
  - Available free online at http://linyun.info/phd-grinding.pdf

8.7.2 Career and Professional Development

- **A PhD Is Not Enough!** by Peter J. Feibelman
  - Guide to building a successful scientific career after PhD
  - Covers networking, job hunting, grant writing, and career planning
  - Practical advice on transitioning from student to professional researcher
- **Getting What You Came For** by Robert Peters
  - Comprehensive guide to choosing and succeeding in graduate school
  - Covers everything from selecting advisors to defending your dissertation
  - Includes strategies for time management and maintaining mental health
- **Deep Work** by Cal Newport
  - Strategies for focused work in a distracted world
  - Particularly relevant for research and writing
  - Techniques for maximizing cognitive capabilities
- **The 7 Habits of Highly Effective People** by Stephen Covey
  - Time management and personal effectiveness
  - Principles applicable to research and collaboration
  - Framework for long-term success and life balance

8.7.3 Online Resources and Blogs

- **The Illustrated Guide to a PhD** by Matt Might
  - Visual explanation of what a PhD really means
  - Helps maintain perspective during challenging times
  - Widely shared and appreciated by PhD students worldwide
- **PhD Comics** by Jorge Cham
  - Humorous take on graduate student life
  - Provides comic relief and community feeling
  - Addresses common PhD experiences and challenges
- **Nature Career Columns**
  - Regular articles on research careers and PhD life

  - Advice from established researchers
  - Current perspectives on academic job market

- **Academia Stack Exchange**
  - Q&A platform for academic life questions
  - Community wisdom on common PhD challenges
  - Practical solutions to specific problems

Reading Recommendation

Start with **The PhD Grind**—it's short, free, and will give you realistic expectations about the PhD journey. Read it before starting your PhD or during your first year to better understand what lies ahead.

8.8 Resource Channels Worth Your Attention

Attention Is the Scarce Resource

Information is not scarce; attention is. A handful of high-signal channels, read at fixed times, beats fifty feeds checked all day. The lists below are starting points—keep only what you actually use.

8.8.1 Papers and Research News

- **Hugging Face Daily Papers**: a community-upvoted daily list of new papers; good for a 10-minute morning scan.
- **alphaXiv**: arXiv papers with discussion threads, plus a personalized feed.
- **Scholar alerts**: in Semantic Scholar or Google Scholar, follow 10–20 key authors and 3–5 landmark papers. New citations of a landmark paper are often the best map of where a subfield is moving.
- **CCF DDL**: conference deadlines with countdowns in AoE time, so deadlines stop sneaking up on you.

8.8.2 Long-Form Interviews and Podcasts

- **Dwarkesh Podcast**: long, technical interviews with researchers and founders.
- **Latent Space**: an AI engineering newsletter and podcast on agents, models, and infrastructure.
- **Training Data (Sequoia)**: founders and researchers on building AI products.

Read Interviews, Don't Just Listen

A two-hour interview is easier to digest as text. Use an AI tool to turn the video into a transcript with chapters and timestamps, then read it at your own pace and jump back to the original for the parts that matter. Keep the concrete examples, not only the conclusions—examples are what you remember and reuse.

8.8.3 Technical Blogs and Learning by Doing

- **Lil'Log by Lilian Weng**: deep, well-referenced surveys of ML topics.
- **Andrej Karpathy's blog**: clear first-principles explanations, including his PhD survival guide.
- **Anthropic Engineering**: practical write-ups on building agents and tooling.
- **BlogrXiv**: an editorially curated index of technical AI writing, organized by topic.
- **AI Systems Playground**: interactive labs where you learn how LLMs, training, inference, and agents work by operating them.

A Presentation Checklist

Before every presentation, verify:

- ☐ Title slide has all required information
- ☐ Outline matches actual content
- ☐ All figures have labeled axes
- ☐ All figures have descriptive captions
- ☐ Experimental setup is clearly explained
- ☐ Results are connected to hypotheses
- ☐ Acknowledgments are included
- ☐ Presentation is within time limit
- ☐ Slides are numbered
- ☐ References are properly cited

B Figure Checklist

For every figure, ensure:

- ☐ Title clearly states what's shown
- ☐ X-axis is labeled with units
- ☐ Y-axis is labeled with units
- ☐ Legend explains all symbols/colors
- ☐ Font size is readable when projected
- ☐ Color scheme is accessible
- ☐ Error bars/confidence intervals shown
- ☐ Caption explains key findings

C Research Tools and Resources

C.1 Recommended Software

Category	Tool	Purpose
Version Control	Git, GitHub	Code management
Writing	LaTeX (Overleaf), Markdown	Documents and papers
Data Visualization	Matplotlib, Seaborn, Plotly	Creating figures
Diagram Tools	Mermaid, PlantUML	Flowcharts and diagrams
Note Taking	Notion, Obsidian, Roam	Research notes

Table 5: Essential research tools for CS and AI researchers

C.2 Terminal Setup Recommendation

Upgrade Your Terminal Experience: iTerm2 + Oh My Zsh

For a significantly improved development experience, we strongly recommend replacing the default macOS Terminal with:

- **iTerm2**: A powerful terminal emulator
- **zsh**: Modern shell (default on macOS)
- **Oh My Zsh**: Framework for managing zsh configuration
- **Powerlevel10k**: Beautiful and informative theme

Key Advantages:

- The terminal interface looks amazing — it shows path, time, git status, conda environment, etc. all by default
- Supports tabs and pane splits, so you don't need to open multiple windows
- Smooth syntax highlighting and auto-completion — feels super nice

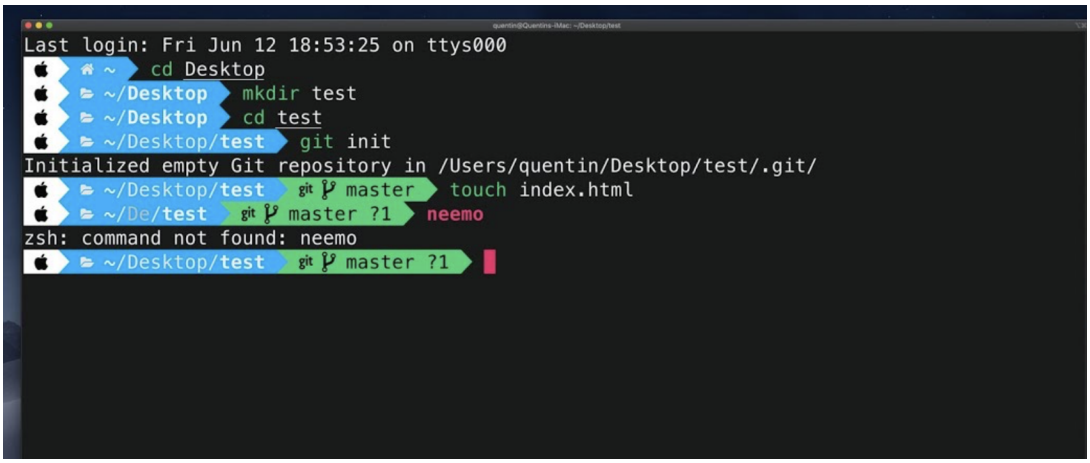


Figure 1: iTerm2 with Powerlevel10k theme showing git status, conda environment, and more

C.3 Visualization and Diagramming Tools

Advanced Tools for Research Visualization

C.3.1 Excalidraw

Website: https://excalidraw.com/

Excalidraw is a virtual whiteboard tool that enables you to create beautiful hand-drawn style diagrams effortlessly:

- **Perfect for:** Quick architectural diagrams, flowcharts, and conceptual illustrations
- **Key Features:**
  - Hand-drawn aesthetic that's perfect for informal presentations
  - Real-time collaboration for team brainstorming
  - Export to PNG, SVG, and clipboard

  - Works entirely in the browser — no installation needed

- **Research Applications:**
  - System architecture sketches during design phase
  - Algorithm flowcharts for presentations
  - Brainstorming session visualizations
  - Quick diagrams for group meetings

C.3.2 PlotNeuralNet

Repository: https://github.com/HarisIqbal88/PlotNeuralNet

PlotNeuralNet is a LaTeX-based tool specifically designed for creating publication-quality neural network architecture diagrams:

- **Perfect for:** Research papers, thesis figures, and technical presentations
- **Key Features:**
  - Creates professional 3D neural network visualizations
  - Fully customizable layers and connections
  - Consistent styling across all diagrams
  - Generates LaTeX/TikZ code for easy integration
- **Research Applications:**
  - CNN/RNN/Transformer architecture illustrations
  - Paper and thesis figures that meet publication standards
  - Technical documentation of model architectures
  - Poster presentations at conferences
- **Pro Tip:** Start with the provided examples and modify them to match your specific architecture — it's much easier than building from scratch!

C.4 System Resource Monitoring and Performance Optimization

Essential Skills for Efficient Development

As researchers working with data processing and deep learning, monitoring system resources and utilizing parallel processing are critical skills that can dramatically improve your productivity and code efficiency.

C.4.1 System Resource Monitoring

Why Monitor Resources?

- Identify bottlenecks in data processing pipelines
- Detect memory leaks and inefficient code early
- Optimize GPU utilization for deep learning tasks
- Prevent system crashes from resource exhaustion
- Validate that parallel processing is working effectively

Essential Monitoring Tools:

- **htop**: Interactive process viewer showing real-time CPU usage per core
- **nvtop**: GPU monitoring for NVIDIA cards (GPU utilization, memory, temperature)
- **gpustat**: Simple command-line GPU status viewer
- **btop**: Modern resource monitor with beautiful graphs and real-time updates
- **nvidia-smi**: Built-in NVIDIA GPU monitoring tool

C.4.2 Parallel Processing for Efficiency

Serial vs Parallel Processing Comparison:

```
Serial Processing:      [Task 1] -> [Task 2] -> [Task 3] -> [Task 4]
                       Time: ================================

Parallel Processing:    [Task 1] [Task 2]
                       [Task 3] [Task 4]
                       Time: ================

Result: ~4x speed improvement (ideal case)
```

Performance Comparison Example:

Task	Serial Time	Parallel Time (4 cores)	Speed-up
Data preprocessing	100s	28s	3.6x
Model training (CPU)	240s	65s	3.7x
Batch inference	60s	18s	3.3x
Image processing	180s	48s	3.8x

Table 6: Typical performance gains from parallelization

Practical Implementation Examples:

- **Python**: Use `multiprocessing`, `concurrent.futures`, or `joblib`
- **NumPy/Pandas**: Vectorized operations instead of loops
- **Deep Learning**: Batch processing, DataLoader with `num_workers` $>$ `0`
- **Data Processing**: Transform `map()` $\rightarrow$ `parallel_map()`

C.4.3 Best Practices and Monitoring Guidelines

What to Monitor During Development:

- **CPU Usage**: Aim for >80% utilization across all cores when parallel processing
- **Memory Usage**: Watch for memory leaks; avoid swapping to disk
- **GPU Utilization**: Should be >90% during deep learning training
- **I/O Wait**: High I/O wait indicates data loading bottlenecks

Common Parallel Processing Pitfalls:

- I/O bottlenecks limiting parallel gains
- Python's GIL (use multiprocessing instead of threading for CPU tasks)
- Overhead from creating too many small parallel tasks
- Insufficient memory causing thrashing

Quick Reference Commands:

Listing 3: System Monitoring Commands

```
# Monitor CPU usage
htop                    # Interactive CPU monitor
top -u                  # Alternative CPU monitor

# Monitor GPU (NVIDIA)
nvidia-smi -l 1       # Updates every 1 second
nvtop                 # Interactive GPU monitor
gpustat -i 1          # Simple GPU status
```

```python
# Python parallel example
from multiprocessing import Pool

def process_func(item):
    # Your processing logic here
    return item * 2

with Pool() as p:
    results = p.map(process_func, data_list)
```

Listing 4: Python Parallel Processing Example

Pro Tips:

- Always benchmark before and after implementing parallelization
- Monitor resources during the ENTIRE training process, not just at the start
- For deep learning: use `DataLoader(num_workers=4-8)` for faster data loading
- Check if your bottleneck is CPU, GPU, or I/O before optimizing

AI Disclosure

Some text and links in this document were compiled with AI assistance. Please forgive any broken links.